SUBJECT 3

Industrial Defect Detection

A plain-English explanation and practical step-by-step project roadmap

Project in one sentence: Build a quality-control assistant that looks at photos of manufactured parts, highlights suspicious defects, and recommends ACCEPT or REJECT—then prove which AI approach works best when you have very few defective examples.

What the project is really asking: Factories usually have lots of examples of normal products but very few examples of defective ones. Can an AI learn what “normal” looks like and spot something unusual, even if it has never seen that exact defect before? At what point is that approach better than training an AI on labelled defect examples?

1. Understand the three approaches

You will compare three families of methods. Think of them as three different inspectors looking at the same products.

Approach	How it works (plain English)	Example methods
Classical image processing	Compare an image with a reference or use image patterns/rules to spot unusual regions. This is your simple, explainable starting point.	Reference subtraction, Otsu thresholding, LBP/GLCM features, SVM or Random Forest
Supervised learning	Show the model labelled examples: “this is a defect” and “this is normal”. It learns to recognize defect types it has been shown.	YOLO (required supervised comparison); Mask R-CNN is an advanced option
Unsupervised anomaly detection	Train mainly or only on good products. The model learns their normal appearance. A new image that looks unusual gets a high anomaly score and a heatmap.	PatchCore and PaDiM (recommended); autoencoder is advanced

Simple example: Imagine inspecting 1,000 clean metal parts and only 5 scratched parts. A supervised model has only 5 defect examples to learn from. An anomaly detector can learn the appearance of clean parts and flag a new scratch as unusual. It may also flag harmless reflections, so its mistakes must be studied.

2. What the finished system should do

1. Take a photo of a manufactured part.

2. Prepare the image so lighting, size, noise and position are more consistent.

3. Run a detector (classical, YOLO, or PatchCore/PaDiM).

4. Show where the problem may be using a box, mask or heatmap.

5. Make a decision: ACCEPT or REJECT, using a threshold chosen on validation data.

6. Explain the result with a confidence/anomaly score and, where possible, the reason for an error.

The application is a project demonstration, not a certified industrial safety system.

3. A practical roadmap: do the work in this order

Phase 1 — Fix the scope and organise the project

• Use the dataset specified by your teacher. The brief mentions MVTec AD for the standard route and VisA as an advanced option. Do not mix datasets casually: choose the level and categories with your supervisor.

• Start with one or two object categories, not all categories. Pick categories with enough normal images and annotated defects.

• Create a GitHub repository, a shared experiment log and a fixed train/validation/test split.

• Keep the test set untouched until final evaluation. All methods must be compared on the same test images.

• For supervised training, reserve the allowed defect examples for training. Never let test defects leak into training.

Phase 2 — Explore the data before modelling

• Count normal and defective images for each category and defect type.

• Open a sample of images and masks. Check whether labels match the visible defects.

• Measure defect size in pixels and note tiny defects, reflections, dust and orientation changes.

• Create a small visual gallery: normal examples, each defect type, and difficult examples.

• Record image dimensions and whether the dataset provides pixel-level masks.

Phase 3 — Build one shared preprocessing pipeline

Preprocessing means preparing every image in a consistent way before inspection. Implement it as reusable functions so you can turn operations on and off.

• **Crop/locate the part:** remove irrelevant background where appropriate. SAM can isolate the object in the advanced route.

• **Normalize:** make image sizes and pixel ranges consistent.

• **Improve contrast:** try CLAHE, especially when details are hard to see.

• **Denoise:** try a median or Gaussian filter, but check that tiny scratches are not erased.

• **Align to a reference:** correct small rotations or shifts using ORB features plus homography, or phase correlation. This matters for reference subtraction.

• Save before/after examples and measure whether each operation helps. Do not assume more preprocessing is always better.

Phase 4 — Make a simple classical baseline

Before using advanced AI, create a basic reference method. Compare a part with a normal reference image, use Otsu thresholding to separate unusual pixels, and optionally describe texture using LBP/GLCM followed by SVM or Random Forest. This gives you a baseline to beat and helps reveal whether the dataset is easy or difficult.

Phase 5 — Train the unsupervised models

• Train PatchCore and PaDiM using normal training images only, following the Anomalib examples/configuration for your chosen dataset.

• Run them on validation and test images. Save both the image-level anomaly score and the pixel-level heatmap.

• Choose an anomaly threshold using normal validation images. The brief suggests starting around the 99th percentile of normal validation scores, then considering business costs.

• Inspect heatmaps: does the model highlight the actual scratch/crack, or just a reflection or background?

Phase 6 — Train the supervised model

• Convert defect masks to bounding boxes in YOLO format if the dataset supplies masks.

• Train YOLO on the permitted supervised training defects. Begin with the full available training-defect set (100%).

• Save predictions and model settings. Keep the test set identical to the one used by the other approaches.

• Mask R-CNN is an advanced extension, not the first thing to implement. Get YOLO working first.

Phase 7 — Run the key experiment: the rare-defect curve

This is the scientific heart of the project. It answers the title question.

YOLO training condition	Meaning	What to compare
100% defects	All permitted training defect examples	How well supervised learning works when labels are plentiful
10% defects	Only one tenth of those examples	How quickly performance falls as labels become scarce
1% defects	Very few defect examples	Whether anomaly detection becomes more useful
0% defects	No defective examples used to train YOLO	Use the unsupervised methods as the no-defect-label comparison

> For each non-zero percentage, make 3 different random selections and retrain YOLO. Keep the same test set. Plot performance against the percentage of defect examples. The “break-even point” is where PatchCore/PaDiM perform better than YOLO under your chosen metric. Do not invent this point: calculate it from your results.

Phase 8 — Try synthetic defects (after the core comparison works)

Copy/paste real defect patches onto normal images to create extra labelled training examples. Add suitable augmentations. Compare YOLO at 1% and 10% defect availability with and without synthetic examples. Check each defect type separately: synthetic scratches may help with scratches but not necessarily with dents or stains.

Phase 9 — Choose ACCEPT/REJECT using costs

A model produces a score; a factory needs a decision. Set the threshold on validation data, not on the test set. Count two kinds of mistakes:

• **False negative:** a defective part is accepted and may reach the customer. This can be very costly.

• **False positive / false rejection:** a good part is rejected, wasting product or inspection time.

Try several thresholds and calculate the total cost using an explicit cost assumption. If shipping a defective part costs 10 times as much as rejecting a good part, the system should generally be more cautious. Show a cost-versus-threshold graph and report the false-rejection rate at a fixed 99% defect-detection target, as suggested by the brief.

Phase 10 — Analyse errors and build the demo

• Create a gallery of false positives and false negatives. Group causes such as reflections, dust, rotation/orientation, tiny defects and poor alignment.

• Build a small Streamlit or Gradio app: upload image → show original → show anomaly heatmap/box → show score → ACCEPT/REJECT.

• Prepare at least one normal example, one correctly detected defect and one explained false positive for the demonstration.

4. How to evaluate the methods

Metric	What it tells you	Direction
Image AUROC	Can the method rank defective parts above normal parts across thresholds?	Higher is better
Pixel AUROC	Can it identify anomalous pixels?	Higher is better

Metric	What it tells you	Direction
PRO score	How well anomaly regions overlap defect regions, giving regions similar importance	Higher is better
F1 and confusion matrix	At the chosen threshold, how many parts are correctly/incorrectly accepted or rejected?	Higher F1 is better
False rejection rate at 99% detection	How many good parts are rejected while detecting 99% of defects?	Lower is better
Inference time (ms/image)	Can the method keep up with the factory line?	Lower is better

Important: Accuracy alone is misleading when defects are extremely rare. A system that calls every part “normal” can appear accurate while missing the defects. Always report AUROC and operational error rates, not accuracy alone.

5. Suggested division of work for a pair

Member A — Classical & anomaly detection	Member B — Supervised & application
Explore defect categories and image statistics; implement CLAHE, denoising and alignment; build classical baseline; train PatchCore/PaDiM; evaluate AUROC/PRO and prepare heatmap/error figures.	Prepare the split and YOLO annotations; train YOLO at 100%, 10%, 1% and 0%; test synthetic defects; measure inference time; build Streamlit/Gradio demo.
Work together: agree on the same split and test set, cost-based threshold, interpretation, report, recommendation and presentation. Both members should be able to explain every method.	**Work together:** use branches and pull requests, review each other's changes, update CONTRIBUTIONS.md, and meet weekly for 30 minutes.

6. Recommended project folder

Keep the code organised from the beginning. A practical structure based on the project brief:

pieces-defects-anomalies/

■■■ README.md # setup and how to reproduce results

■■■ requirements.txt # fixed package versions

■■■ CONTRIBUTIONS.md # who did what

■■■ configs/ # model and rarity experiment settings

■■■ data/ # downloaded data; do not upload images to GitHub

■■■ scripts/ # download, split, annotation conversion

■■■ src/

■ ■■■ preprocessing/ # crop, CLAHE, denoising, alignment

■ ■■■ classical/ # reference, Otsu, LBP/GLCM, SVM

■ ■■■ supervised/ # YOLO, synthetic defects

■ ■■■ unsupervised/ # PatchCore, PaDiM

■ ■■■ evaluation/ # AUROC, PRO, costs, timing

■■■ results/ # CSV tables, plots, error gallery

■■■ app/ # Streamlit or Gradio

■■■ docs/ # report and slides

7. A realistic order of priorities

Priority	Finish this first	Why
Must-have 1	Dataset, fixed split, data exploration and preprocessing	Everything depends on trustworthy data
Must-have 2	Classical baseline + PatchCore/PaDiM + YOLO at 100%	Establish the basic comparison
Must-have 3	YOLO rarity experiment: 10%, 1%, 0%; 3 random draws	This is the central research result
Must-have 4	Threshold/cost analysis, error gallery and demo	Turns model scores into an industrial decision
Optional / advanced	Mask R-CNN, autoencoder, SAM, extensive synthetic defects	Add only after the core experiment works and time allows

8. Final deliverables and presentation

• **GitHub:** reproducible code, README, fixed requirements and random seeds, CONTRIBUTIONS.md, and a v1.0 release/tag. Do not commit dataset images or large model weights.

• **Technical report:** 15–20 pages excluding appendices, covering industrial context, data, pipeline, methods, protocol, results, interpretation, limitations and recommendation.

• **Results:** classical vs supervised vs unsupervised comparison; rarity curve; cost/threshold curve; false-positive/false-negative gallery.

• **Demo:** upload an image → show defect heatmap → show ACCEPT/REJECT. Keep a 1–2 minute backup video.

• **Slides:** 6–10 PowerPoint slides. In the 5–10 minute defence, both members speak. A useful structure is problem (1 min), pipeline (1 min), results (2–3 min), demo (1–2 min), recommendation/limits (1 min).

Dates in the project brief: final submission is proposed for Friday 12 March 2027 at 23:59 (to be confirmed); defence is Monday 15 March 2027. Check these dates with the teacher.

9. Common traps to avoid

• Do not train or tune on the test set. Choose thresholds and parameters using validation data only.

• Do not compare methods on different test images. Keep one common test set.

• Do not use accuracy as the only metric when defects are rare.

• Do not confuse an anomaly score with a final ACCEPT/REJECT decision; you must choose and justify a threshold.

• Do not assume every highlighted anomaly is a real defect. Reflections, dust and normal orientation changes can trigger false alarms.

• Do not start with every advanced model. Get the core comparison working first, then extend it.

10. What your final conclusion should answer

Your project should finish with a practical recommendation, supported by your measurements. For example: “When the factory has plenty of labelled defects, YOLO is the better choice; when labelled defects fall below a measured level, PatchCore/PaDiM becomes preferable. At the selected operating point, the system detects a chosen proportion of defects while rejecting a measured proportion of good parts.” Replace every placeholder with your actual experimental results.

Source basis: the teacher's Subject 3 project specification (Détection de pièces défaillantes — focus données rares et détection d'anomalies). This guide explains that brief in practical language and adds an implementation order; it does not replace the teacher's instructions about dataset, project level or dates.